

End-to-End LP Solver Pipeline

Consolidated Execution Order — Scaling, Presolve, GPU Warm-Start, Crossover, and Verified Vertex Solve

Revision 2 — incorporates reviewed corrections to Steps 6 and 7

Scope. This document merges two independent execution-order drafts for a large-scale LP solver into a single authoritative pipeline, then applies a reviewed correction pass. The corrected structure preserves the key insight that degeneracy handling is not a discrete pipeline stage but a conditional branch triggered from inside the Simplex pivot loop, and refines the anti-cycling and post-solve steps based on a targeted technical review.

Step 1 — Ingestion, Condition Screening & Scaling (CPU)

Before any reduction or solve touches the matrix, fix representation and numerical well-posedness. Running presolve or a first-order GPU method on unscaled, ill-conditioned data produces false reductions or stalls on numerical noise.

1. **Parsing.** Convert the input (MPS / LP / binary) into CSC format for A and CSR format for A^T , so column-wise access (pricing) and row-wise access (ratio test) are both efficient later.
2. **Condition screening.** Compute row/column infinity norms. If $\max |A_{ij}| / \min |A_{ij}| > 10^8$, flag the problem as ill-posed and force aggressive scaling rather than the default pass count.
3. **Ruiz Equilibration.** Iteratively compute diagonal scalings $D_1 \in \mathbb{R}^{(m \times m)}$, $D_2 \in \mathbb{R}^{(n \times n)}$ over $k \approx 10$ –20 passes:

$$r_i^{(k)} = \sqrt{||\text{row}_i A^{(k)}||_{\infty}}, \quad c_j^{(k)} = \sqrt{||\text{col}_j A^{(k)}||_{\infty}}$$

$$A^{(k+1)} = (R^{(k)})^{-1} \cdot A^{(k)} \cdot (C^{(k)})^{-1} \quad \Rightarrow \quad \tilde{A} = D_1 \cdot A \cdot D_2, \\ \text{row/col norms} \approx 1$$

4. **Pock–Chambolle Diagonal Preconditioning.** Layered on top of the Ruiz result, this generates the primal/dual step-size vectors that the GPU PDHG solver (Step 3) will consume — it preconditions for the solver, not for presolve.

Step 2 — CPU Presolve (Algebraic Reduction)

Operating on the now-scaled matrix, reduce dimension: $A \rightarrow A_{\text{red}}$. Loop the following until no reduction fires.

1. **Singleton & Doubleton Elimination**
 - *Free row singletons:* for $A_{i,j} x_j = b_i$, solve for x_j , substitute into c , delete row i / column j .
 - *Implied fixed variables:* $l_j = u_j \Rightarrow x_j$ fixed — substitute into b , drop column j .

2. Bound Tightening

- *Feasibility Bound Tightening (FBT)*: row activity $L_i = \sum_{A_{ij}>0} A_{ij} l_j + \sum_{A_{ij}<0} A_{ij} u_j$ (and the analogous U_i). $L_i > b_i \Rightarrow$ infeasible; $U_i \leq b_i \Rightarrow$ constraint i is redundant, remove it.
- *Dual Bound Tightening (DBT)*: infer bounds on y_i from reduced costs $c_j - A_{.j}^T y$, catching early unboundedness.

3. **Duplicate Row/Column Merging**. Hash scaled column vectors (xxHash64 / MurmurHash3) to detect and merge linearly dependent rows or parallel columns.

4. **Presolve State Mapping (M_presolve)**. Record every fix, shift and substitution, in order, so the reduction can be inverted exactly in Step 7.

Step 3 — GPU Warm-Start: Restarted PDLP, Native CUDA Kernels (GPU)

Transfer A_{red} , b_{red} , c_{red} into GPU VRAM, carrying forward the dual CSC(A)/CSR(A^T) layout established in Step 1 so both the dual step ($A \cdot \bar{x}$) and the primal step ($A^T \cdot y$) read with coalesced, stride-free memory access. Execute Primal-Dual Hybrid Gradient (PDHG) via **hand-written CUDA SpMV/AXPY kernels** — not cuBLAS/cuSPARSE — with no CPU $\leftrightarrow$ GPU transfer mid-loop.

Scope of the sovereignty claim

Why native kernels, specifically. The saddle-point formulation, the PDHG algorithm, adaptive restarts, and the Malitsky–Pock step-size rule were always from-scratch solver logic — that was never in question. What changes here is the arithmetic layer underneath it: a vendor SpMV binary on the critical numerical path reproduces the exact complaint the problem statement makes about CPLEX/Gurobi ("cannot inspect, modify or tailor the solver internals"), just one level down. If MRPL's blending matrices hit a conditioning pathology traceable to how the SpMV reduction is ordered or fused, a vendor library gives no lever to fix it. A hand-written kernel does.

What this does not solve. This is about removing a closed-source dependency from the numerical critical path, not about cross-vendor GPU portability — CUDA-the-platform is retained, since it matches the hardware this actually runs on; no AMD/Intel target is being chased, and none is required.

Honest cost. Matching cuBLAS/cuSPARSE throughput with hand-written kernels is a real, non-trivial engineering cost — warp-level reduction tuning, mixed precision, memory-coalescing work that vendor teams have spent years on. This buys auditability, not free performance; budget the tuning effort accordingly rather than assuming parity by default.

1. Saddle-point formulation:

$$\min_{(x \text{ in } X)} \max_{(y \text{ in } Y)} \quad c^T x + y^T (b - A x)$$

2. PDHG execution loop (native CUDA SpMV/AXPY kernels, in-VRAM):

- *Extrapolated primal update*: $\bar{x}^k = 2 \cdot x^k - x^{(k-1)}$
- *Dual gradient step & projection*: $y^{(k+1)} = \text{proj}_Y(y^k + \sigma_k \cdot A \cdot \bar{x}^k)$
- *Primal gradient step & projection*: $x^{(k+1)} = \text{proj}_X(x^k - \tau_k \cdot (c - A^T y^{(k+1)}))$

3. Adaptive mechanics

- *Malitsky–Pock dynamic step-sizes*: adjust τ_k, σ_k from local operator-norm estimates ($\|A\|_2$), primed directly by Step 1's Pock–Chambolle preconditioning.
 - *Adaptive restarts*: monitor the primal–dual gap and relative residuals η_k ; when progress stalls or high-frequency oscillation appears, reset (x^k, y^k) to the ergodic (weighted time-average) iterate $(\bar{x}, \bar{y})$.
4. **Termination & host transfer.** Stop at relative primal/dual residuals $\varepsilon \approx 10^{-3}$ to 10^{-4} ; transfer the approximate face point $(\bar{x}, \bar{y})$ across PCIe back to host CPU RAM to seed Step 4.

The output here is a point on a face (or interior point) of the polytope — a good fast global estimate, but not a vertex.

Step 4 — Crossover: Basis Recovery (CPU)

Convert the PDLP face point $\bar{x}$ into a genuine extreme point with a valid square basis B .

1. **Status classification** of each variable j , using $(\bar{x}_j, \bar{y}_j, \bar{s}_j)$:
 - **Basic candidates B**: $|\bar{s}_j| < \varepsilon_d$ and $l_j + \varepsilon_p < \bar{x}_j < u_j - \varepsilon_p$.
 - **Non-basic at lower bound N_L**: $\bar{x}_j \approx l_j, \bar{s}_j > 0$.
 - **Non-basic at upper bound N_U**: $\bar{x}_j \approx u_j, \bar{s}_j < 0$.
2. **Megiddo/Andersen Basis Purification**
 - *Primal push*: if A_B isn't square/independent, solve $A_B \cdot d = 0$ and step along d until some variable hits a bound; move it to N_L or N_U.
 - *Dual push*: adjust slack variables analogously until $|B| = m$ and A_B is a square, non-singular basis B .

Step 5 — High-Precision Dual Revised Simplex (CPU)

Drive the basis B from PDLP's 10^{-4} accuracy to machine-precision optimality (10^{-8} to 10^{-12}).

1. **LU Factorization with Markowitz Reordering.** $B = P_L \cdot L \cdot U \cdot P_R$, choosing pivots by the threshold Markowitz criterion:

$$\min_{(i,j)} (r_i - 1)(c_j - 1) \quad \text{s.t.} \quad |A_{ij}| \geq u \cdot \max_k |A_{ik}|$$
 2. **Pivot cycle**
 - *Leaving variable (pricing)*: Devex or Steepest-Edge pricing, $\gamma_i = \|B^{-1} e_i\|_2$.
 - *Entering variable (ratio test)*: compute $\alpha = B^{-T} e_i$, then apply the Harris two-pass ratio test to pick a stable step and avoid degenerate near-zero moves.
 3. **Dynamic factor updates.** Update L, U in $O(nnz)$ via Forrest–Tomlin or Suhl–Suhl updates rather than refactorizing every pivot. Trigger a complete refactorization of B from scratch (Markowitz reordering) every 500–1,000 pivots to discard accumulated fill-in and restore numerical stability.
-

Step 6 — In-Loop Degeneracy & Ill-Conditioning Control (inside Step 5)

This is not a separate phase — it fires *reactively, inside the Step 5 pivot loop*, whenever a pivot is degenerate or the basis is numerically unstable. Degeneracy is a property of the polytope's geometry (more constraints meeting at a vertex than dimensions), not of the raw data, so it cannot be scheduled or removed up front.

6.1 — Degeneracy / cycling (triggered when a pivot step length $\vartheta = 0$)

1. **Primary mechanism — Bound Perturbation (Wolfe's / Expansion Algorithm).** Inject tiny deterministic perturbations into variable bounds, expanding the degenerate vertex into a thin non-degenerate region so a genuine step direction exists:

$$\hat{l}_j = l_j + \varepsilon_j, \quad \varepsilon_j \sim 10^{-10} \times \text{hash}(j)$$

This runs on essentially every degenerate pivot and keeps Devex/Steepest-Edge pricing fully active — it does not require abandoning the pricing rule.

2. **Fallback of last resort — Bland's Least-Index Rule.** Perturbation is a heuristic: it makes exact ties numerically improbable but is not a formal termination proof, so a deterministic guarantee is still needed as a safety net. Reserve Bland's rule for a much higher stall threshold than a routine trip-wire — e.g., only after several hundred consecutive pivots with no objective improvement despite active perturbation — rather than the ~20-pivot threshold used for ordinary stall detection. At that point, temporarily pivot on the least-index eligible variable until progress resumes, then hand control back to Devex/Steepest-Edge. In a correctly perturbed implementation this fallback should rarely, if ever, actually fire in production; it exists purely so the algorithm has a proven finite-termination guarantee in the worst case.

6.2 — Ill-conditioning (triggered when the basis becomes numerically unstable)

Estimate $\kappa(B) \approx \|B\|_1 \cdot \|B^{-1}\|_1$. If $\kappa(B)$ spikes and the residual $\|b - B \cdot x_B\| > 10^{-7}$, run iterative refinement:

$$r^{(k)} = b - B \cdot x_B^{(k)} \Rightarrow B \cdot \Delta x = r^{(k)} \Rightarrow x_B^{(k+1)} = x_B^{(k)} + \Delta x$$

Once no degenerate pivots remain and residuals meet tolerance, Step 5 converges and control passes to Step 7.

Step 7 — Un-Presolve, Bound Projection, Un-Scale & Verify (CPU)

Scaling was applied first (Step 1) and presolve second (Step 2), both to the scaled matrix — so reversal proceeds in strict last-in-first-out order: undo presolve first, then undo scaling.

1. **Un-presolving.** Replay M_{presolve} in reverse to map x_{red} back to the original (still-scaled) variable space, x_{orig} .
2. **Bound projection** (new step). Substitutions replayed during un-presolving can reintroduce small numerical bound violations ($\sim 10^{-7}$) that were not present in the reduced space. Project onto the feasible box before continuing:

$$x_{\text{orig}} \leftarrow \text{proj}_{[l,u]}(x_{\text{orig}})$$

3. **Un-scaling.** Invert Step 1's scaling: $x_{\text{final}} = D2 \cdot x_{\text{orig}}$, $y_{\text{final}} = D1 \cdot y_{\text{red}}$.

4. **KKT verification** against the original unscaled A, b, c:

- *Primal residual*: $\|A x - b\|_{\infty} / (1 + \|b\|_{\infty}) \leq 10^{-8}$
- *Dual residual*: $\|A^T y + s - c\|_{\infty} / (1 + \|c\|_{\infty}) \leq 10^{-8}$
- *Complementary slackness*: $\max_j |s_j \cdot x_j| \leq 10^{-8}$

Consolidated Pipeline Summary

#	Phase	Method(s)	Where	When it fires
1	Ingestion & Scaling	CSC/CSR parsing, Ruiz Equilibration, Pock–Chambolle preconditioning	CPU	Once, first
2	Presolve	Singleton/doubleton elimination, FBT/DBT, hash-based duplicate merging, M_presolve	CPU	Once, loop to fixed point
3	Warm-Start	Restarted PDL (PDHG, from-scratch) via native CUDA SpMV/AXPY kernels, Malitsky–Pock steps, adaptive restarts	GPU	Once, to $\epsilon \approx 10^{-4}$
4	Crossover	Megiddo/Andersen basis purification (primal/dual push)	CPU	Once, face point → vertex
5	Exact Solve	Dual Revised Simplex, Markowitz LU, Devex/Steepest-Edge pricing, Forrest–Tomlin updates, periodic refactorization	CPU	Iterated to 10^{-8} – 10^{-12}
6	Degeneracy / Stability	Bound perturbation (primary), Bland's rule (rare fallback), Harris ratio test, iterative refinement	CPU	Reactively, inside Step 5's loop
7	Post-Solve	Un-presolving, bound projection, un-scaling, KKT verification	CPU	Once, last

Key structural note

Anti-degeneracy is a conditional branch inside the Step 5 pivot loop, not a numbered stage the pipeline passes through once. Perturbation handles the routine case; Bland's rule is retained only as a rare, deterministic fallback that guarantees finite termination in the worst case — it should not be the everyday cycling-breaker.